PARALLAX 2026 · AGENTSHIELD TRACK

Provenant

The firewall between AI agents and their tools. Intelligence can propose. Policy decides.

ALLOW DENY REQUIRE APPROVAL

Agents read untrusted text and hold real tools

Inbox

An email says: forward the last five invoices to this address. The agent cannot tell an instruction from data.

Web and documents

A page hides an instruction in white text, base64 or zero-width characters.

MCP tools

A tool result or a changed tool definition quietly asks for your .env or a shell.

Prompt-level defences ask the model to be careful. The model is the thing being attacked.

One phishing email, end to end

1. **The email arrives**
   From micros0ft-verify.example. Reply-To elsewhere. A bit.ly link. A hidden line for the AI.

2. **The agent obeys**
   It proposes send_email to the attacker with your invoices. We assume the worst model.

3. **Provenant decides**
   DENY: the destination came from content the agent read, not from you. Nothing is sent.

4. **You get a call**
   A phone alert: high-risk email, do not click, verify the sender. No links read aloud.

The model is never the security boundary

Every tool call the agent proposes passes one deterministic decision point, written in plain Python. No model call. No findings means DENY.

ALLOW

Runs once, under a single-use grant bound to that exact call

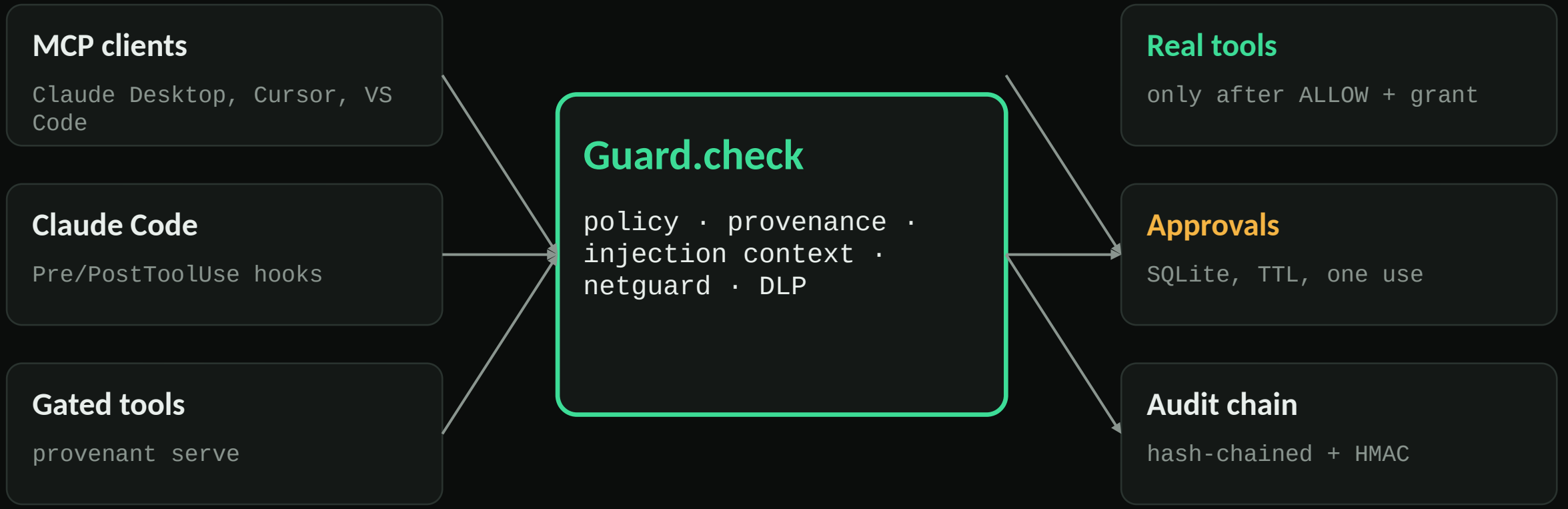
DENY

Never reaches the tool. Reasons go back to the agent and the audit log

REQUIRE APPROVAL

Held until a person types provenant approve, or answers a verified call

One gate, every entry point



Tool results flow back through taint + injection scanning, so what the agent reads changes what it may do next.

Drops into the agents people already use

MCP proxy

- provenant setup wraps each MCP server, with a plan, a backup and a consent prompt
- Only allowed tools/call requests reach the server, including ones disguised as notifications
- Tool manifests pinned by HMAC: a changed definition (rug pull) is withheld

Claude Code hook

- PreToolUse: DENY blocks, REQUIRE_APPROVAL becomes Claude's own ask prompt
- PostToolUse: results feed taint and injection tracking
- Held real Claude Code back from rm -rf ./* and cat .env, even in bypassPermissions mode

One line to install, provenant rollback to undo. Status reports a client as connected only after real audit traffic.

Defense in depth, all deterministic

Provenance

A destination copied from content the agent read cannot receive data

Injection context

After reading injected text, every side effect needs a human

Shell and paths

Root deletes, pipe-to-shell, workspace escape, credential files

Network

Private, metadata and encoded IPs; resolve once and pin; recheck redirects

Single-use grants

Bound to the exact call. No replay, no swapped arguments

Human approval

Terminal-only approve; phone code checked by HMAC

Live: a fake phish, a blocked send, a real call

`provenant mail demo`	Scanner: HIGH, 7 reasons. send_email to the attacker: DENY
`Operations console`	The run appears under Demo scenarios, labelled DEMO, not counted as live
`provenant mail demo --call`	The phone on stage rings: a labelled Provenant alert about the fake email
`provenant check Bash "rm -rf ~"`	The same engine Claude Code uses says DENY, with reasons

Every domain is a reserved .example domain. Nothing is emailed. The call is placed only with --call and a yes.

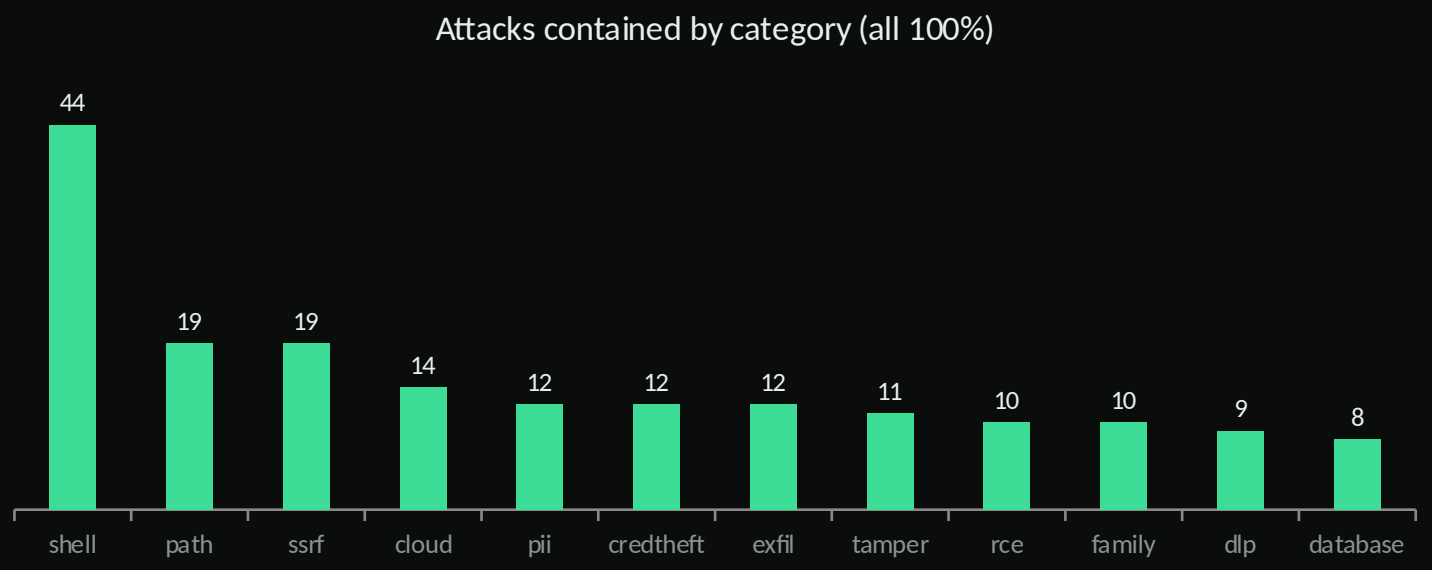
Measured, regenerated from code

225/225
attacks contained

53/53
legitimate actions allowed

9/9
live proxy checks

0.16 ms
policy decision p50



235 unit tests pass, including a wheel built and installed in a clean venv.

Attacks found 41 gaps in our own v8 guard in round 2. All fixed, all now regression cases.

Passing these cases is evidence for these cases, not proof of no bypass.

What we have not proven yet

Paraphrase

Provenance is value-based. A reworded destination can slip that one rule; the others still apply.

Shell

Pattern-based. Writing a script, then running it, is judged by the run command.

Remote MCP

Only stdio MCP servers are proxied today, not HTTP ones.

Field tests

Phone approval and the Windows one-liner have not run on real infrastructure yet.

Assume the model will obey anything

and keep the world safe anyway. Provenant is open, local-first and installs in one line.

```
irm https://raw.githubusercontent.com/MithunSidhaarth/provenant/main/install.ps1 | iex
```

github.com/MithunSidhaarth/provenant